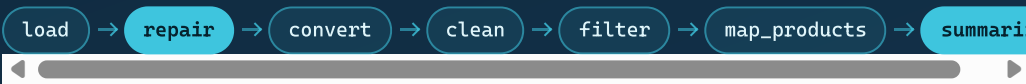


System Ticket Data Summary

Raw ticket exports, distilled into customer stories

A validated LangGraph pipeline that loads, repairs and cleans system ticket data, generates per-customer storytelling summaries with an LLM, and serves a business insights dashboard through Streamlit - with every data repair fingerprinted, reported, and audited.



42

raw tickets parsed and validated against a 38-column contract

19

shifted rows detected by fingerprint and repaired, with a printed report

18

storytelling summaries per run - one per customer per product

0

invented or omitted tickets across the full summary audit

§ Contents

01 Overview and architecture

02 The data: columns and quality issues

03 Implementation: data preprocessing

04 Implementation: category mapping

05 Implementation: summary generation

06 Code documentation

07 User guide: the Streamlit app

08 Business insights (bonus analysis)

01 Overview and architecture

The task: convert a raw system-ticket text file into per-customer, per-product storytelling summaries generated by an LLM, presented in a Streamlit web app, with a data-quality-safe preprocessing pipeline and a business insights dashboard.

The whole pipeline is a **LangGraph graph**: every step is a node, and one `invoke` call carries a file from raw text to finished summaries:



COMPONENT	ROLE
<code>notebooks/ticket_summary.ipynb</code>	Prototype and teaching artifact: each step explained, explored line by line with visible output, then assembled into a function
<code>pipeline.py</code>	The reusable module: all step functions plus the compiled LangGraph graph
<code>app.py</code>	Streamlit app: upload, preprocessing report, AI summaries, insights dashboard
<code>.env</code>	Secrets and model choice (<code>GOOGLE_API_KEY</code> , <code>GEMINI_MODEL</code>); git-ignored, never hard-coded

Design rule 1 Validate, never repair silently. Malformed input raises a clear error with line numbers. The one repair that is performed (the column shift, §3.2) is fingerprint-verified and reports every affected ticket number on every run.

Design rule 2 One job per function. Each step is a small function used identically by the notebook, the graph and the app - changing a grouping or a prompt touches one place.

02 The data: columns and quality issues

The raw file is a comma-separated table: line 1 holds 38 column names, every following line is one ticket (42 in the sample; mixed English and German).

2.1 Column groups

GROUP	COLUMNS	MEANING
Identifiers	ORDER_NUMBER, ORDER_ID	Public ticket reference (001-.../24) and internal id
Customer	CUSTOMER_NUMBER, CUSTOMER_COUNT	Anonymized customer id (123 / 456 / 789); contract size
Timing	ACCEPTANCE_TIME, COMPLETION_TIME	Opened / closed timestamps (MM/DD/YYYY HH:MM); other *_TIME_* columns are workflow copies or empty
Classification	ORDER_TYPE, ORDER_CLASS, PROCESSING_STATUS, SERVICE_CATEGORY	Short Ticket vs Task; all closed; category code (HDW, NET, KAI, KAV, GIGA, VOD, KAD; NTF/SOP excluded)
Problem	ORDER_DESCRIPTION_1/2/3, NOTE_MAXIMUM	Device, function, symptom hierarchy; agent's note of the customer report
Handling	PLANNING_GROUP_KB, COMPLETION_RESULT_KB, COMPLETION_NOTE_MAXIMUM, CAUSE	Team code (TSCW2/TSCKT/TSCS2); the fix; closing note; diagnosed root-cause code
Empty	ORDER_UNIT_ID, ASSIGNMENT_*, others	Completely empty in this export; dropped during cleaning with a printed report

2.2 Quality issues discovered - and how each is handled

ISSUE	SYMPTOM	HANDLING
UTF-8 BOM	Invisible character glued to the first column name; <code>df["ORDER_NUMBER"]</code> raised KeyError	Read with <code>encoding="utf-8-sig"</code>
Shifted rows	19 of 42 rows written with one missing N/A placeholder: every value from ADDITIONAL_ORDER_DESCRIPTION onward sat one column too far left (team code in the note field, the fix in the team field)	Fingerprint detection + reported repair (§3.2)
Literal "N/A" text	Missing values stored as the string N/A, polluting counts and LLM input	Converted to real missing values during cleaning
Mixed languages	German rows (Kurtzticket, Kabelrouter, keine Verbindung)	Category codes are language-neutral; the LLM translates narratives; analytics normalize device names

03 Implementation: data preprocessing

3.1 Load and validate

Parses with `csv.reader`. Accepts files **with or without a header line**:

- Line 1 equals the standard 38-column header - header present.
- Line 1 matches the order-number pattern `\d{3}-\d+/\d{2}` (e.g. 001-0671177/24) and has 38 fields - headerless file; the standard header is applied and the app reports it.
- Anything else raises: wrong field count ("different ticket scheme?"), unrecognizable line 1 ("wrong file?"), empty file, or a header with no tickets.

```
def load_raw_tickets(source):
    ...
    parsed = [r for r in csv.reader(io.StringIO(text)) if r]
    first = parsed[0]
    if first == EXPECTED_COLUMNS:          # line 1 is exactly our header
        header, data, had_header = first, parsed[1:], True
    else:                                  # headerless file
        if len(first) != len(EXPECTED_COLUMNS):
            raise ValueError(f"This file has {len(first)} fields per row, expected 38 ...")
        if not ORDER_NUMBER_PATTERN.fullmatch(first[0]):
            raise ValueError("Line 1 is neither the ticket header nor a ticket row ...")
        header, data, had_header = list(EXPECTED_COLUMNS), parsed, False

    bad = [(line, len(r)) ... if len(r) != n_cols]      # ragged-row check
    if bad:
        raise ValueError(f"Incomplete tickets - ... (line, fields): {bad}")
```

The schema itself is one verbatim string (`EXPECTED_HEADER`): a single source of truth copied from the ticket system's export and split into the 38 names.

3.2 Repair the shifted rows

The fingerprint: support-team codes (TSCW2, TSKCT, TSCS2) are only ever legal in `PLANNING_GROUP_KB`. A team code sitting in `NOTE_MAXIMUM` proves the row slid one column left. The repair is two list operations, applied only to fingerprinted rows:

```

for r in rows:
    if r[note_idx] in TEAM_CODES:      # team code where the note belongs = shifted row
        r.insert(add_idx, "N/A")      # re-insert the placeholder → rest slides right
        r.pop()                       # drop the padding beyond the last column
        repaired.append(r[0])
print(f"Repaired {len(repaired)} shifted tickets: {repaired}")

```

Why this is safe Nothing is invented: every value already exists in the row and only moves back under its correct header. The removed tail is always padding (verified: N/A or empty in all 19 rows). The repair is idempotent - on already-repaired data it finds zero fingerprints. Without it, the LLM would read team codes as customer notes for 19 of 32 tickets.

3.3 Convert

The repaired table is persisted as `data/tickets_converted.csv` and `.xlsx` - the format conversion the task requires, done after repair so the files carry corrected columns.

3.4 Clean

```

out = out.apply(lambda s: s.str.strip())      # stray spaces
out = out.replace({"N/A": pd.NA, "": pd.NA})  # literal N/A → real missing values
out["ACCEPTANCE_TIME"] = pd.to_datetime(out["ACCEPTANCE_TIME"], format="%m/%d/%Y %H:%M")
out["COMPLETION_TIME"] = pd.to_datetime(out["COMPLETION_TIME"], format="%m/%d/%Y %H:%M")
out["RESOLUTION_MINUTES"] = (out["COMPLETION_TIME"] - out["ACCEPTANCE_TIME"]).dt.total_seconds() /
empty_cols = out.columns[out.isna().all()]    # 100%-empty columns only
out = out.drop(columns=empty_cols)            # drop is printed, never silent

```

Dates use the default `errors="raise"`: a garbage timestamp stops the run loudly. The empty-column check uses `.all()`, so a column with even a single real value always survives; after the repair, 10 columns are genuinely empty and dropped.

3.5 Filter

Only the predefined categories HDW, NET, KAI, KAV, GIGA, VOD, KAD are kept (`isin` mask); NTF inquiries and SOP activations are excluded. Result: 32 of 42 tickets, sorted chronologically - order matters for storytelling.

04 Implementation: category mapping

PRODUCT	CATEGORIES	NOTE
Broadband	KAI, NET	Mapping exactly as specified by the task document
Voice	KAV	
TV	KAD	
GIGA	GIGA	
VOD	VOD	
Hardware	HDW	HDW passes the category filter but has no product in the task document; it is kept as its own group rather than silently dropped

```
CATEGORY_TO_PRODUCT = {"KAI": "Broadband", "NET": "Broadband", "KAV": "Voice",  
                        "KAD": "TV", "GIGA": "GIGA", "VOD": "VOD", "HDW": "Hardware"}  
out["PRODUCT"] = out["SERVICE_CATEGORY"].map(CATEGORY_TO_PRODUCT)
```

05 Implementation: summary generation

Summaries are generated **per customer per product** - each narrative describes one real customer's history with one product. For the sample file: 3 customers x 6 products = 18 summaries, returned as `{customer: {product: text}}`.

5.1 Ticket serialization

The LLM receives each group's tickets as compact chronological lines; only non-missing fields are included - which is why cleaning produced real missing values first:

```
- 001-0671177/24 | 2024-11-05 11:00 | cat=HDW | issue: Cable Router / WLAN / Slow  
  | note: WLAN slow | resolution: WLAN settings optimized / OK
```

5.2 The output contract: schema, validation, evaluation

The summary is not free text - it is a validated contract, built in four layers:

- **Schema** - a Pydantic model (five phases, each with `ticket_numbers` and `narrative`), enforced natively by the API during generation (`response_schema`): missing sections or wrong types are impossible. There is deliberately **no timeframe field** - what the model does not author, it cannot hallucinate.
- **Semantic validators** - every ticket reference must match the order-number pattern; "N/A" or padding values are rejected on sight (absence is an empty list, never a value).
- **Reference evaluation** - two checklists against the group's real tickets: cited-but-not-real (invented) and real-but-not-cited (omitted). Violations trigger a corrective retry that names the exact mistakes and hands over the true ticket list; persistent failures ship with a visible "unverified" badge, never silently.
- **Computed timeframes** - each section's displayed period is looked up from the verified tickets' real timestamps by pandas. A displayed date cannot be hallucinated because the model never writes one.

Guarantee The reference check runs live on every generation - each of the 18 summaries carries a status (**verified / corrected / unverified**) shown in the app. Validators are inspectors, not editors: a flawed answer is rejected and regenerated by the model, never silently trimmed or patched by our code. Narrative prose remains grounded (the model sees only the ticket lines) and auditable via its per-section citations - verified where verifiable, grounded where not.

5.3 Execution

The chain `STORY_PROMPT | llm | StrOutputParser()` is wrapped in `.with_retry(stop_after_attempt=3)`: the Gemini endpoint intermittently drops connections, and one hiccup must not kill an 18-call run. The graph's `summarize` node loops the groups and reports progress per group. The model name lives in `.env` only (`gemma-4-31b-it` for development with its high free quota, `gemini-3.5-flash` for final runs) - switching models is a one-line change, which proved itself when Google retired gemini-2.5-flash mid-project.

06 Code documentation

- **pipeline.py** - every function carries a docstring stating its contract; inline comments mark only non-obvious constraints (why utf-8-sig, why the fingerprint is reliable, why `.all()` not `.any()`, why retries exist).
- **notebooks/ticket_summary.ipynb** - every step is documented three times: a markdown cell explaining the logic in plain words, exploratory cells running each line separately with visible output, then the assembled function. Debug trails (the BOM incident) are kept and labeled for learning value.
- **app.py** - section banners per tab; all findings under charts are computed from the loaded data at render time, never hardcoded text.

6.1 pipeline.py function reference

FUNCTION	CONTRACT
<code>load_raw_tickets(source)</code>	Path, file-like, text or bytes to <code>(header, rows, had_header)</code> ; raises <code>ValueError</code> on any malformed input
<code>find_shifted_orders(rows, header)</code>	Order numbers of fingerprinted (shifted) rows, for UI reporting before repair
<code>repair_shifted_rows(rows, header)</code>	Insert/pop repair in place; prints repaired ticket numbers; idempotent
<code>convert_and_save(df, out_dir)</code>	Writes CSV + XLSX; returns paths
<code>clean_tickets(df)</code>	Strip, N/A to missing, parse dates (raise on garbage), <code>RESOLUTION_MINUTES</code> , drop 100%-empty columns with report
<code>filter_categories(df)</code>	Keep allowed categories, sort chronologically
<code>map_products(df)</code>	Add <code>PRODUCT</code> column from the mapping table
<code>format_tickets(df)</code>	<code>DataFrame</code> to compact chronological text lines for the LLM
<code>get_llm()</code>	<code>ChatGoogleGenerativeAI</code> from <code>.env</code> (<code>GEMINI_MODEL</code>); raises if the key is missing
<code>evaluate_story(story, group)</code>	Reference checklists: invented + omitted tickets vs the group's data; empty list = proven
<code>summarize_group(group, llm, ...)</code>	Generate, evaluate, corrective retries; returns (story, status)
<code>section_timeframe / render_story</code>	Data-computed timeframes; validated object to five-section Markdown
<code>summarize_products(df, llm, on_progress)</code>	Per customer x product: {markdown, status, story} entries, transient-error retries
<code>build_pipeline(llm, on_progress)</code>	Compiles the full LangGraph graph (nodes added one by one, linear edges)

07 User guide: the Streamlit app

7.1 Setup (once)

```
uv sync                                # or: pip install -r requirements.txt
# .env in the project root:
GOOGLE_API_KEY=your-key-here          # https://aistudio.google.com/apikey
GEMINI_MODEL=gemma-4-31b-it
```

7.2 Run

```
streamlit run app.py                  # browser opens at http://localhost:8501
```

7.3 Using the app

1. **Data source:** the bundled sample dataset loads automatically - no upload needed. The sidebar's "Active data file" panel shows which file is in use (with a raw download for inspection). Optionally upload your own `.txt` or `.csv`, with or without the header line (a banner reports when the standard header was applied automatically).
2. **Preprocessing report:** a data-quality panel lists any repaired shifted tickets; the metric row shows raw count, filtered count, products and average resolution time. A clearly worded error appears instead if the file is invalid (wrong scheme, wrong file, empty, broken rows).
3. **Data tab:** the converted table (with CSV download) and the cleaned, product-mapped table.
4. **AI Summaries tab:** click *Generate summaries*, watch per-group progress, then pick a **customer** and a **product** to read that story - each summary shows its verification badge (verified / corrected / unverified) and an aggregate reference-check line. Download the complete set as a formatted **PDF** (one page per customer x product) or as Markdown.
5. **Insights tab:** an executive scorecard (repeat rate, first-contact resolution, escalation rate, resolution time, contacts per issue) followed by the business analysis (§08), with product and customer filters - all findings recompute for the current selection. Extra views (most common fixes, FCR by product, team workload, root-cause codes, volume timelines) sit in a clearly labeled expander.

7.4 Notebook usage

Open `notebooks/ticket_summary.ipynb` and Run All: Steps 0-6 run without any API key; Step 7 onward needs `GOOGLE_API_KEY` in `.env`. Step 10 prints the 18 summaries; Step 11 reproduces the insights charts cell by cell.

08 Business insights (bonus analysis)

The analysis is structured as the three questions every support dataset must answer. Each chart in the app ends with a finding computed live from the data.

Q1

What breaks the most?

Pareto per device (German names normalized) + tickets per product - where the volume comes from.

Q2

Do fixes hold?

Repeat-contact chains per customer x product, with days between contacts and each chain's final resolution.

Q3

Which customers need attention?

Tickets per customer by product + first vs repeat contacts - the churn-risk view.

Above the questions sits an **executive scorecard** computed from the filtered data: repeat-contact rate 44%, first-contact resolution 67%, escalation rate 28%, average resolution 33 minutes, 1.8 contacts per issue.

44%

OF ALL TICKETS

Q1 FINDING

One device is almost half the support volume

14 of 32 analyzed tickets are the cable router - all three customers, the same device, within the same two weeks. No other issue area exceeds 3 tickets.

Action: one targeted fix (firmware push, batch swap, self-service WLAN reset) attacks 44% of the entire queue at once.

100%

OF REPEAT
CHAINS ENDED IN
ESCALATION

Q2 FINDING

Repeated remote fixes never held

Measured hold times inside the router chains: 2 to 25 hours per remote fix (best case 6 days, then total device failure). Not one repeated remote fix cured the problem - every chain still ended in a device replacement or a technician visit.

Action: pilot an escalation trigger on the second contact for the same issue - and re-measure the cure rate per contact number at production scale before fixing the threshold.

11

CONTACTS FROM
ONE CUSTOMER IN
TWO WEEKS

Q3 FINDING

Every customer is a repeat caller

All three customers contacted support 10-11 times inside two weeks, and 44% of those contacts were repeats about a product whose previous ticket had already been closed as resolved. First-contact resolution: 100% for four products, 0% for Hardware - one device destroys the KPI.

Action: treat heavy repeat callers as churn risks, not workload - contact them proactively once the device fix ships, before they consider cancellation.

SCALE CAVEAT 42 tickets, 3 customers, 2 weeks. The findings are real in this file and the method is production-ready, but staffing views (weekday x hour heatmap, daily volume, customer timelines) stay illustrative until fed real volume - the app labels them accordingly.